Few-Shot, Label-Free Fabric Defect Inspection for Networked Seaming Stations via Confidence-Gated Edge Offloading

Nasiba Palvannazirova, Ayaz Waseem, Nikita Lobasevich, Petr Butovskiy, and Miralam Allamov

***Abstract*—Automated fabric inspection relies on supervised detectors that need many labelled defects per fabric and GPU training, which does not fit plants where networked seaming stations process short runs of many materials. We present FabricInspect-IoT, a three-tier Industrial Internet of Things system for few-shot, label-free surface inspection. From ten defect-free frames, the system fits a 6-kB Gabor–colour Gaussian model that runs on a single station core and a ResNet-18 nearest-neighbour model on a CPU edge server; both derive their decision thresholds by leave-one-out scoring of the defect-free frames, without any defect example. A confidence gate decides clear cases at the station and offloads only uncertain frames over MQTT, and fabric profiles are distributed to all stations as retained messages. On the MVTec AD textures, the edge model reaches F1 0.935 with 4.3 % false alarms on four of five categories, and the gate retains 99 % of the edge-only F1 while offloading half of the frames and cutting station-only false alarms from 28 % to 8 %. A trace-driven simulator, validated against a live testbed with a Mosquitto broker, shows that one CPU edge server meets a 200-ms decision deadline for 24 instead of 16 stations on Wi-Fi with the gate, reduces uplink traffic per station by 39 %, and that on 1-Mbit/s uplinks only locally decided frames meet the deadline. New fabrics are learned in about one second and reach all stations within 15 ms. Code and data processing scripts are publicly available.**

***Index Terms*—Industrial Internet of Things (IIoT), edge computing, computation offloading, MQTT, anomaly detection, fabric defect detection, few-shot learning.**

I. Introduction

FABRIC inspection is one of the most labour-intensive steps of textile manufacturing. Human inspectors are slow, tire quickly, and miss a substantial fraction of defects, which has motivated two decades of research on automated visual inspection [1], [2]. The state of the art is dominated by supervised deep detectors and segmenters—improved two-stage detectors [3], efficient single-stage detectors [4], [5], YOLO-family models on edge hardware [6], [7], transformer segmenters with Gabor priors [8], and open-world formulations [9]. These models reach high accuracy on the fabrics they were trained on, but they require hundreds to thousands of labelled defect instances per fabric family and a GPU-centred training workflow; the scarcity of labelled defects and the cost of retraining for new materials are repeatedly identified as the main obstacles to deployment [2].

The obstacle is acute on the *shop floor of small and medium-sized garment and technical-textile plants*, where a number of seaming or joining machines process short runs of many different fabrics. Our motivating installation is a set of ultrasonic seaming stations, each controlled by an ESP32 micro-controller and an Android application (FabricScope) and equipped with a USB camera that already identifies the fibre and weave before the seam parameters are chosen. A fabric typically stays on a machine for minutes, defects are rare, and there is neither time nor expertise to label data or retrain a model whenever the material changes. Such a deployment is naturally an Industrial Internet of Things (IIoT) system [10]: many low-cost camera nodes, a shared plant network, and at most one modest edge server. The inspection function must therefore (i) learn a new fabric from a few seconds of defect-free material, (ii) set its own decision threshold without defect examples, (iii) run within the frame budget of an inexpensive station node, and (iv) scale to many stations without saturating the uplink or the edge server.

Unsupervised anomaly detection, which learns only from normal samples, satisfies (i) and is an active direction for fabrics [11]–[13]; on generic benchmarks, training-free methods that reuse ImageNet features [14]–[16] are particularly strong. However, these works report threshold-free metrics with hundreds of training images on a GPU, and are silent on (ii)–(iv). Conversely, edge-intelligence research [17]–[19] and fog-based manufacturing inspection [20] study where to run inference, but assume supervised models trained offline. To our knowledge, no prior work addresses few-shot, label-free surface inspection *as a networked IIoT service*.

This paper fills that gap with *FabricInspect-IoT*, an open-source three-tier system (station node, edge server, profile registry over MQTT) and an empirical study of its design. Our contributions are:

1) A **few-shot, label-free inspection service** in which a fabric is learned from k defect-free frames and two complementary models are built at once: a 5–6 kB Gabor–colour Gaussian model that runs on the station, and a ResNet-18 nearest-neighbour model (PatchCore-lite) that runs on the edge. Decision thresholds are derived

N. Palvannazirova and A. Waseem are with Electronic Science and Technology, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: nasiba@bupt.edu.cn; 2024090002@bupt.cn).

N. Lobasevich is with the School of Economics and Management, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: lnikbest@gmail.com).

P. Butovskiy and M. Allamov are with Technological Machines and Apparatus, Tashkent Institute of Textile and Light Industry, Tashkent, Uzbekistan (e-mail: gtnz1071980@gmail.com; miralam3843678@gmail.com).

Corresponding author: Petr Butovskiy.

Code, configuration, and raw results: https://github.com/nasin0764-bit/FabricInspect-IoT.

by leave-one-out (LOO) scoring of the k shots, without any defect example.

2) **Confidence-gated offloading**: the station decides clear cases locally and offloads only frames whose local score falls in an uncertainty band around its LOO threshold. Because both thresholds are label-free, the gate needs no tuning data.
3) A **profile-distribution mechanism** based on retained MQTT messages, which makes a newly learned fabric available to every station within milliseconds, and an analytical model of uplink load and edge utilisation that predicts how many stations an edge server can support.
4) A **reproducible evaluation** on the MVTec AD textures [21], [22] covering few-shot accuracy ($k = 1 \ldots$all), label-free thresholding, and robustness, together with a live testbed with a production MQTT broker [23] and a trace-driven simulator validated against it, which scales the system to 64 stations on emulated Wi-Fi and constrained uplinks and reports end-to-end latency, deadline misses, uplink traffic, and edge utilisation.

The remainder of the paper is organised as follows. Section II reviews related work. Section III presents the architecture and protocol, and Section IV the detectors, the thresholds, the gate, and the load model. Section V describes the experimental setup, Section VI reports the results, and Section VII discusses limitations before Section VIII concludes.

II. Related Work

A. Fabric Defect Detection

Classical approaches characterise fabric texture with statistical, spectral, and model-based descriptors [1]. Gabor filters are a recurring choice because their joint spatial–frequency localisation matches the periodic structure of woven and knitted fabric; filter banks [24] and optimised filters [25] separate flaws from regular texture without defect data, and Gabor priors are still used to stabilise modern networks [8]. Supervised deep detectors dominate recent work: cascaded feature pyramids [3], efficient detectors with L-shaped pyramids [4], metaheuristic-tuned networks [26], fine-grained space–depth compensation [27], and dual-image transformers for periodic lace [28]. Deployment-oriented studies report MobileNetV2-SSDLite on Jetson Nano with cloud model updates [5], YOLOv8 on a real factory data set [7], and YOLOv11 on Jetson Orin with MQTT reporting [6]; all require labelled defects for every product family.

B. Unsupervised and Few-Shot Anomaly Detection

Learning only from defect-free material avoids labelling. For fabrics, DMGLAD combines teacher–student distillation with global–local reasoning [11], PNEMFD synthesises feature-level anomalies [12], UMDNet transfers unbiased knowledge from natural images [13], and OW-DLN incrementally learns unknown defect classes [9]. These methods train networks for hours on hundreds of images. Training-free alternatives reuse ImageNet features [29]: a single multivariate Gaussian of deep features [14], per-position Gaussians (PaDiM) [15], and nearest-neighbour patch memories (PatchCore) [16]. They are fitted in seconds, which makes them candidates for per-fabric learning at the machine, but their behaviour with very few shots, without labelled thresholds, and on CPUs has not been characterised for textiles.

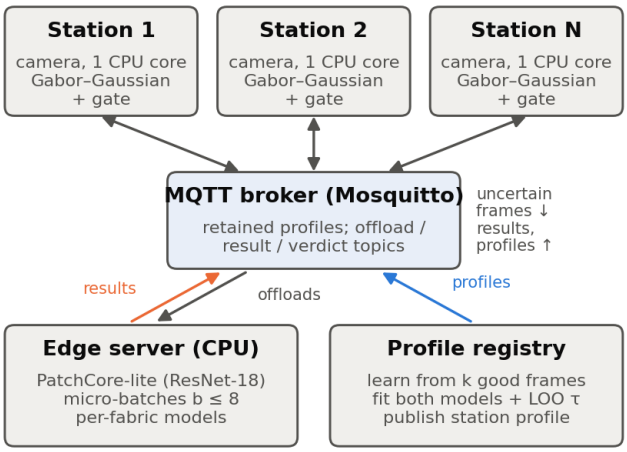


Fig. 1. FabricInspect-IoT. The registry learns a fabric once from k defect-free frames, publishes the compact station model as a retained MQTT message, and installs the deep model on the edge. Stations score every frame and offload only uncertain ones.

C. Edge Intelligence and Offloading in the IIoT

Edge computing moves computation close to data sources to reduce latency and backhaul traffic [17], [30]. For deep inference, the main design levers are partitioning a network between device and server (Neurosurgeon [31], Edgent [19]), early exits that stop computation once confidence is sufficient (BranchyNet [32]), and fog-assisted inspection pipelines in factories [20]; Zhou *et al.* [18] survey the field. These approaches assume a supervised model whose confidence is calibrated on labelled data. Our gate is in the spirit of early exiting, but between two *different* label-free models on two tiers, with the exit band derived from LOO statistics of normal shots only. MQTT [33] is the de-facto publish/subscribe protocol in IIoT; we use its retained messages to distribute fabric models.

D. Material Identification at the Machine

Beyond surface defects, sorting and processing lines identify fibre composition, e.g., by near-infrared spectroscopy [34], [35], and X-ray scattering reveals internal fibre shifts in fabric-reinforced composites [36]. In our installation the station already classifies the material to set ultrasonic seam parameters; FabricInspect-IoT adds the missing surface-quality check in front of the seam, reusing the same camera.

III. System Architecture

Fig. 1 shows the three tiers. *Station node:* each seaming machine carries a USB camera looking at the fabric in front of the seam line and a small computer (in our installation the machine's Android controller; in the experiments one

CPU core). It holds only the station profile of the current fabric, scores each frame with the Gabor–Gaussian detector (Section IV-A), and applies the confidence gate (Section IV-D). *Edge server:* one plant-level x86 machine without GPU subscribes to the offloaded frames of all stations, groups pending requests into micro-batches of up to $b_{\max} = 8$, scores them with the PatchCore-lite model of the corresponding fabric (Section IV-B), and returns the result. *Profile registry:* when an operator learns a new fabric, the registry fits both models from the same k frames, computes both thresholds, publishes the station profile as a *retained* MQTT message, and stores the edge profile on the server. Because the broker keeps the last retained message per topic, a station that starts, reboots, or switches to that fabric receives the model immediately on subscription.

All communication uses MQTT 3.1.1 [33] through a standard broker (Mosquitto [23]) on four topics: `fi/profile/<fabric>/gabor` (station model, QoS 1, retained), `fi/st/<id>/offload` (JSON header plus 256×256 JPEG crop at quality 85, QoS 1), `fi/st/<id>/result` (JSON score and verdict, QoS 1), and `fi/st/<id>/verdict` (informational, QoS 0).

In *learn* mode the operator slowly moves defect-free material under the camera while a frame is grabbed every 0.4 s until k frames (default 10) are collected. In *inspect* mode each frame is centre-cropped, scored, and shown with a heat-map whose above-threshold regions are outlined; every decision is logged and defective frames are saved for review. A desktop GUI, a command-line interface, and the three IoT roles share one detector module, so the numbers in this paper are produced by the code an operator runs. The implementation uses Python, OpenCV, SciPy, Eclipse Paho, and PyTorch (CPU) with public ImageNet ResNet-18 weights [29].

IV. Detection, Thresholding, and Offloading

All detectors receive a colour image resized to 256×256 pixels, are fitted on k defect-free images only, and return an anomaly map $A(\mathbf{x})$ that is bilinearly up-sampled from the feature grid and smoothed with a Gaussian ($\sigma = 4$ px). The image score is $s = \max_{\mathbf{x}} A(\mathbf{x})$.

A. Station Model: Gabor–Gaussian

The image is converted to CIE L*a*b*. The lightness channel L is filtered with quadrature Gabor pairs at wavelengths $\lambda \in \{4, 8, 16\}$ px and orientations $\theta \in \{0°, 45°, 90°, 135°\}$ (envelope $\sigma = 0.56\lambda$, aspect ratio 0.5), giving 12 energy maps

$$E_{\lambda,\theta} = \sqrt{\left(L \star g^{\mathrm{e}}_{\lambda,\theta}\right)^2 + \left(L \star g^{\mathrm{o}}_{\lambda,\theta}\right)^2}. \tag{1}$$

All 24 correlations are computed with one FFT of the reflect-padded image and 12 inverse FFTs of precomputed complex spectra, which is numerically equivalent to spatial filtering (max. relative deviation $< 10^{-6}$) but about $3\times$ cheaper on one core. For the 12 energies and for L, a^*, b^*, local mean and standard deviation over a 15×15 window are sampled with stride 4, yielding a $D = 30$-dimensional descriptor $\mathbf{f}(\mathbf{x})$ on a 64×64 grid. Because fabric texture is approximately stationary, a *single* Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is fitted to the descriptors of all positions of all shots (at most $2 \cdot 10^5$ randomly chosen vectors), with ridge regularisation $\boldsymbol{\Sigma}_\epsilon = \boldsymbol{\Sigma} + \epsilon\,(\operatorname{tr}\boldsymbol{\Sigma}/D)\,\mathbf{I}$, $\epsilon = 0.01$, and

$$A(\mathbf{x}) = \sqrt{(\mathbf{f}(\mathbf{x}) - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}_\epsilon^{-1} (\mathbf{f}(\mathbf{x}) - \boldsymbol{\mu})}. \tag{2}$$

Sharing one Gaussian across positions means that even a single image supplies 4096 training vectors, so the model is usable at $k = 1$. The profile consists of $D + D^2 = 930$ numbers plus the threshold.

B. Edge Model: PatchCore-lite

Following PatchCore [16], ResNet-18 layer-2 ($128\times32\times32$) and layer-3 ($256 \times 16 \times 16$) features are averaged over 3×3 neighbourhoods, layer 3 is up-sampled to 32×32, and the two are concatenated into 384-dimensional patch descriptors. All descriptors of the k shots form a memory bank that is randomly subsampled to 10 % (at least 1000 entries); a test patch is scored by its Euclidean distance to the nearest bank entry. Random rather than greedy-coreset subsampling keeps fitting essentially instantaneous; we therefore call the method PatchCore-lite. As a third, position-specific reference we also evaluate PaDiM [15] (ResNet-18 layers 1–3, 100 random channels, one Gaussian per position), which is not deployed because its per-position statistics make the profile large and LOO calibration expensive.

C. Label-Free Thresholds by Leave-One-Out

When a new fabric is learned, no defect examples exist. For each shot i the detector is refitted on the other $k-1$ shots and the score s_i^{LOO} of shot i is computed; these scores estimate the score distribution of *unseen* defect-free frames (scoring the training shots with the full model would be biased low—for a nearest-neighbour model, trivially zero). The threshold is

$$\tau = (1 + m) \max_{i=1..k} s_i^{\mathrm{LOO}}, \tag{3}$$

with margin $m = 0.1$ by default. LOO costs k refits on cached features: milliseconds for the Gabor–Gaussian model and well under a second for PatchCore-lite at $k = 10$. We denote the station and edge thresholds by τ_g and τ_p.

D. Confidence-Gated Offloading

Let s_g be the station score of a frame. With $0 < \alpha \le 1 \le \beta$, the station

$$\text{decision} = \begin{cases} \text{OK (local)}, & s_g < \alpha\,\tau_g, \\ \text{DEFECT (local)}, & s_g > \beta\,\tau_g, \\ \text{offload} \;\Rightarrow\; \text{DEFECT iff } s_p > \tau_p, & \text{otherwise}, \end{cases} \tag{4}$$

where s_p is the edge score. $\alpha = \beta = 1$ reduces to station-only inspection; $\alpha \to 0, \beta \to \infty$ to edge-only. Both thresholds come from (3), so the band is expressed in units of the fabric's own normal-score scale and transfers across fabrics without labelled tuning. We fix $\alpha = 0.6, \beta = 1.5$ a priori and report the full (α, β) trade-off in Section VI-C.

E. Load Model

Let N stations capture f frames/s, a fraction o of which are offloaded with mean payload J bytes (plus MQTT/TCP/IP overhead h). The per-station uplink rate and the edge utilisation are

$$R_{\mathrm{up}} = 8fo(J+h), \qquad \rho = \frac{Nfo}{\mu(b)}, \tag{5}$$

where $\mu(b)$ is the edge throughput (frames/s) at batch size b. The edge can serve at most $N_{\max} = \mu/(fo)$ stations; cutting o by gating raises $N_{\max}$ proportionally. The end-to-end latency of an offloaded frame is approximately

$$T \approx t_g + \frac{8(J+h)}{B} + 2d + W_q + t_p(b), \tag{6}$$

with station compute t_g, uplink bit-rate B, one-way delay d, queueing delay W_q, and edge compute t_p; locally decided frames take t_g only. Section VI-E compares (5) with measurements.

V. Experimental Setup

A. Data

Openly downloadable, pixel-annotated fabric-defect benchmarks are scarce; the AITEX database [37] could not be obtained without registration at the time of writing. We therefore use the five texture categories of MVTec AD [21], [22]—carpet, grid, leather, tile, and wood—as stationary-texture proxies for the materials processed on joining machines: carpet is a woven textile, grid a regular mesh, leather a sheet material seamed on the same class of machines, and tile and wood test generality. Each category provides 230–280 defect-free training images and 78–124 test images with pixel masks covering five defect types (e.g., cut, hole, thread, colour, metal contamination, fold, crack). The data set is released under CC BY-NC-SA 4.0.

B. Protocol and Metrics

For each category, k defect-free images are drawn at random from the training split, $k \in \{1, 2, 5, 10, 20, 50, \mathrm{all}\}$, with three random draws (one for "all"). Test images are never used for fitting, thresholding or gating. Threshold-free detection is measured by image-level AUROC; localisation by pixel-level AUROC and by AUPRO up to 30 % false-positive rate [22]. With the label-free thresholds we report the false-alarm rate on good images (FPR), defect recall, precision, and F1. The Gabor–Gaussian descriptor (wavelengths, window, colour statistics) was fixed in a preliminary exploration on the same categories at $k = 20$; to make this visible we report the ablation in Section VI-D. The gate band $(\alpha, \beta) = (0.6, 1.5)$ was fixed before the gate experiments were run.

C. System Experiments

The live testbed runs the actual system: Eclipse Mosquitto 2.1.2 as broker, one edge server process (8 PyTorch threads, micro-batch $b_{\max} = 8$), and station processes restricted to one CPU thread each. Station i inspects the test images of category $i \bmod 5$ in random order at $f = 5$ frames/s, a rate that covers a 50 mm field of view without gaps at seam speeds up to 15 m/min. Each station is fitted with $k = 10$ shots. Uplinks are emulated at the application layer: every station owns a FIFO link with bit-rate B and one-way delay d—"Wi-Fi" ($B = 20$ Mbit/s, $d = 4$ ms) and "constrained" ($B = 1$ Mbit/s, $d = 40$ ms)—and incoming results are delayed by d. A decision is a deadline miss if it is not available at the station within one frame period (200 ms) or never arrives. Three policies are compared: station-only ($\alpha = \beta = 1$), edge-only (thin station, every frame offloaded), and the gated cascade. Because a single laptop cannot host many stations without contention and throttling, scalability is studied with a trace-driven simulator that is validated against short live runs ($N \in \{1, 4\}$, both links); Section VI-E details the procedure. Every timing measurement is preceded and followed by a speed check of the station and edge code, runs in which the host was throttled are discarded, and individual live configurations with signs of transient throttling are flagged.

D. Hardware and Software

All processes run on one host (Intel Core i9-14900HX, 24 cores/32 threads, 32 GB RAM, Windows 11) with Python 3.9, PyTorch 2.8 (CPU), OpenCV 5.0, SciPy 1.13, and Paho-MQTT 2.1; no GPU is used. Restricting stations to one thread approximates a low-cost single-board station node.

VI. Results

A. Few-Shot Detection

Table I lists all metrics at the default $k = 10$; the complete curves for $k = 1 \ldots$all are in the repository. *PatchCore-lite is the most accurate detector at every k*: averaged over the five textures its image AUROC is 0.904 with a single shot, 0.916 at $k = 10$, and 0.977 with the full training set (AUPRO 0.747, 0.786, 0.894). *The stationary Gabor–Gaussian model is the most shot-efficient*: it reaches 0.828 at $k = 1$—better than PaDiM (0.725), whose per-position Gaussians are poorly estimated from few images—and saturates early (0.860 at $k = 10$, 0.872 with all shots). The gap between station and edge model is *fabric-dependent*: on leather, tile, and wood the station model is within 0.07 AUROC of PatchCore-lite, whereas on carpet (0.740 vs. 0.967) and grid (0.674 vs. 0.636) it is not adequate at $k = 10$. Grid is hard for all detectors with few shots and improves with more normal data (PatchCore-lite 0.921 with all 264 images).

Fig. 2 shows typical anomaly maps. Both models localise the carpet thread, the leather cut, and the tile crack, and both miss the small broken cells of the grid. On wood the colour-sensitive station model flags the bleached scratch whereas PatchCore-lite stays below its threshold: the models are complementary rather than nested. This fabric dependence motivates the two-tier design.

B. Label-Free Thresholds

With the LOO threshold (3), PatchCore-lite at $k = 10$ and $m = 0.1$ produces 3.4 % false alarms on good test images

TABLE I
DETECTION AND LOCALISATION WITH $k = 10$ DEFECT-FREE SHOTS (MEAN OF THREE RANDOM DRAWS). GG: GABOR–GAUSSIAN (STATION), PC: PATCHCORE-LITE (EDGE), PD: PADIM. BEST PER METRIC IN BOLD.

	Image AUROC			Pixel AUROC			AUPRO		
Category	GG	PC	PD	GG	PC	PD	GG	PC	PD
carpet	0.740	**0.967**	0.881	0.883	**0.988**	0.972	0.643	**0.933**	0.911
grid	**0.674**	0.636	0.633	0.621	**0.742**	0.676	0.178	**0.333**	0.330
leather	0.984	**0.999**	0.961	**0.993**	0.992	0.989	**0.984**	0.965	0.967
tile	0.919	**0.986**	0.841	0.936	**0.940**	0.820	0.768	**0.817**	0.606
wood	0.981	**0.992**	0.989	**0.932**	0.929	0.931	0.879	0.882	**0.908**
Mean	0.860	**0.916**	0.861	0.873	**0.918**	0.878	0.690	**0.786**	0.744

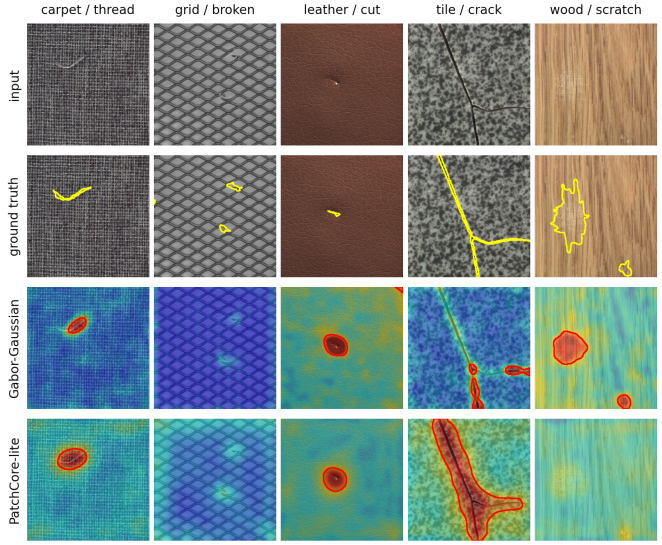


Fig. 2. Anomaly maps of the station (Gabor–Gaussian) and edge (PatchCore-lite) models ($k = 10$). Yellow: ground truth; red: regions above the label-free threshold.

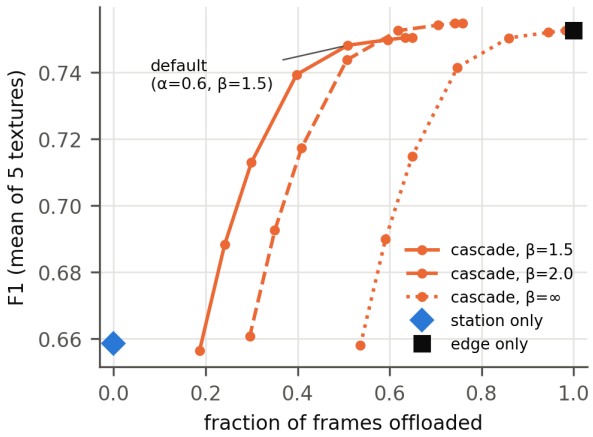


Fig. 3. Accuracy–offload trade-off of the gate ($k = 10$, mean of five textures and three draws); each curve sweeps α from 1.0 (left) to 0.3 (right).

at 71.8 % defect recall (F1 0.753, mean of five textures and three draws). Without margin, recall rises to 78.8 % but false alarms to 24.5 %; $m = 0.2$ removes false alarms entirely at 65.3 % recall, so m is a single, interpretable knob between nuisance stops and escapes. Results are similar for $k = 5$ and 20 (F1 0.755 and 0.754). Grid is the exception: its LOO threshold is so conservative that almost no defect is flagged (F1 0.023); on the other four textures PatchCore-lite reaches F1 0.935 at 4.3 % false alarms and 89.4 % recall. The station model's LOO threshold is far less reliable (28.2 % false alarms at $k = 10$, $m = 0.1$), dominated by leather (95.8 %) and carpet (38.1 %), where ten LOO scores underestimate the range of normal scores. The station threshold is therefore best used to *define* an uncertainty band rather than to issue final verdicts near the boundary.

C. Confidence-Gated Offloading

Fig. 3 shows the accuracy–offload trade-off obtained by sweeping $\alpha \in [0.3, 1]$ for three values of β, and Table II details the a-priori operating point. Station-only inspection never offloads but reaches only F1 0.659 with 28.2 % false alarms; edge-only inspection reaches F1 0.753 at 3.4 % but offloads

TABLE II
PER-FABRIC DECISIONS OF THE THREE POLICIES ($k = 10$, $m = 0.1$, MEAN OF THREE DRAWS). CASCADE USES THE A-PRIORI BAND $\alpha = 0.6, \beta = 1.5$; "OFF." IS THE OFFLOADED FRACTION.

	Station only		Cascade			Edge only	
Category	F1	FPR	F1	FPR	off.	F1	FPR
carpet	0.718	0.381	0.918	0.143	0.81	0.923	0.095
grid	0.090	0.016	0.023	0.000	0.52	0.023	0.000
leather	0.857	0.958	0.963	0.198	0.23	0.985	0.042
tile	0.717	0.000	0.880	0.000	0.68	0.872	0.000
wood	0.911	0.053	0.957	0.053	0.30	0.960	0.035
Mean	0.659	0.282	0.748	0.079	0.51	0.753	0.034

every frame. The cascade with $(\alpha, \beta) = (0.6, 1.5)$ attains F1 0.748—99.3 % of the edge-only value—while offloading 50.9 % of the frames, and cuts the false-alarm rate to 7.9 % (without grid: F1 0.930 vs. 0.935 edge-only and 0.801 station-only). Along the Pareto front, $(0.7, 1.5)$ keeps F1 at 0.739 with 39.8 % offload and $(0.6, 2.0)$ matches edge-only with 61.8 %; the lower bound α is the dominant knob.

The gate adapts to the fabric without any tuning (Table II): it offloads 23 % of leather and 30 % of wood frames, where the station model is strong, but 81 % of carpet frames, where it is weak, and it corrects the station's 95.8 % false-alarm rate on leather to 19.8 %.

D. Resources, Profiles, Ablation, and Robustness

On one core the station model needs 36.1 ms per frame (p95 40.3 ms), i.e., it sustains 27 frames/s, more than five times the inspection rate, without any deep-learning runtime. Its serialised profile is 5.0–6.1 kB, whereas a PatchCore-lite memory bank occupies 1.1–1.3 MB and a PaDiM model 165 MB, which rules PaDiM out for distribution. With eight threads the edge model scores a frame in 16–20 ms including JPEG decoding; micro-batching raises its throughput μ from 64 frames/s at $b = 1$ to 96 frames/s at $b = 8$. Learning a fabric from $k = 10$ frames, including both LOO calibrations, takes 0.55–0.96 s for the station model and 0.17 s for PatchCore-lite. The retained station profile reaches every subscribed station in 10–15 ms on the Wi-Fi link and 93 ms on the constrained link, independently of whether 1, 8, or 32 stations subscribe; a PatchCore-lite profile would need 0.52 s and 10.1 s. A fabric change thus costs seconds of operator time, not a deployment step.

Ablation ($k = 20$): removing the Lab chroma statistics from the station descriptor costs 0.029 image AUROC and 0.065 AUPRO, since colour defects are invisible to a greyscale texture model; removing the Gabor bank costs 0.039 and 0.108. Six orientations instead of four add 0.016 AUROC at 40 % more dimensions and filter cost; we keep four as a compromise between accuracy, compute, and profile size.

Robustness ($k = 20$, test frames perturbed at working resolution, models fitted on clean shots): ± 20 % brightness changes AUROC by at most 0.009 for all detectors, since a global gain shifts good and defective scores alike. Defocus blur ($\sigma = 2$ px) costs 0.08–0.12 for all detectors. Sensor noise ($\sigma = 10$ grey levels) affects the station model most (-0.128, its finest Gabor scale responds to pixel noise) and PatchCore-lite least (-0.048). A change of focus, lighting, or camera should therefore be followed by re-learning the fabric, which takes seconds and needs no labels.

E. System Performance and Scalability

Methodology. Hour-long live sweeps with up to 32 station processes, a broker, and an edge server on one laptop proved unsuitable for timing: under sustained load the host repeatedly entered power-limited states in which all code ran 4–8× slower, and the station processes competed for cores that real stations would not share. We therefore measure the service-time distributions of every component in short, speed-checked runs (station model: 300 frames on one core; edge: 60 batches for each $b = 1 \ldots 8$; JPEG payloads of all test frames) and feed them into a trace-driven discrete-event simulator of Fig. 1: periodic capture, a single-core station, a FIFO uplink per station with the emulated rate and delay, a single batching edge server ($b_{\max} = 8$), and the downlink delay. About 15 % of the edge samples were 5–10× slower than the median because of the same host throttling; samples above twice the median are excluded, and a slower edge is studied explicitly below. The only free parameter, a constant messaging and serialisation overhead per offloaded frame, is calibrated on one live configuration (Wi-Fi, edge-only, $N = 1$): 38.7 ms.

TABLE III
VALIDATION OF THE TRACE-DRIVEN SIMULATOR AGAINST THE LIVE TESTBED (LIVE / SIMULATED, BOTH 12 S AT 5 FRAMES/S; SIMULATION: MEAN OF 20 SEEDS). LATENCY IN MS, MISSES IN %. †LIVE RUN AFFECTED BY TRANSIENT HOST THROTTLING (STATION MEDIAN $> 2\times$ ITS TRACE MEDIAN, OR EDGE UTILISATION $> 3\times$ SIMULATED).

Link	Policy	N	p50	p95	Miss	Edge util.
Wi-Fi†	station-only	1	143 / 36	161 / 40	0.0 / 0.0	0.00 / 0.00
Wi-Fi	station-only	4	53 / 36	75 / 40	0.0 / 0.0	0.00 / 0.00
Wi-Fi	cascade	1	115 / 108	143 / 115	0.0 / 0.0	0.16 / 0.06
Wi-Fi	cascade	4	94 / 99	142 / 115	0.0 / 0.0	0.24 / 0.18
Wi-Fi	edge-only	1	73 / 73	80 / 78	0.0 / 0.0	0.20 / 0.08
Wi-Fi	edge-only	4	96 / 73	143 / 83	0.0 / 0.0	0.55 / 0.32
constr.	station-only	1	46 / 36	153 / 40	0.0 / 0.0	0.00 / 0.00
constr.†	station-only	4	170 / 36	191 / 40	0.8 / 0.0	0.00 / 0.00
constr.†	cascade	1	678 / 402	807 / 498	95.0 / 80.5	0.61 / 0.06
constr.†	cascade	4	679 / 310	1482 / 447	60.0 / 56.1	0.96 / 0.18
constr.	edge-only	1	683 / 667	1016 / 953	100.0 / 100.0	0.15 / 0.07
constr.	edge-only	4	1095 / 290	3556 / 855	100.0 / 100.0	0.80 / 0.31

Validation. Table III compares the simulator with short live runs of the complete system. On Wi-Fi the simulated median latency of the cascade is within 7 ms of the live value and its p95 within 28 ms; for edge-only with $N = 4$ the live p95 is 60 ms higher. Deadline-miss rates agree in every Wi-Fi configuration and, on the constrained link, within 15 percentage points. The live edge was 1.4–2.6× busier than simulated in the unflagged configurations, because on the testbed it shares the CPU with the station processes and the broker; the "edge 2× slower" scenario brackets this case. Four live runs show signs of transient host throttling (marked †).

Scaling on Wi-Fi (Fig. 4). Station-only inspection scales trivially—each decision takes 40 ms (p95) on the station's own core and no uplink is used—but inherits the station model's accuracy (F1 0.659). Edge-only inspection meets the 200-ms deadline for up to $N = 16$ stations (p95 154 ms, 0.3 % misses), at which point the edge is saturated ($\rho = 0.99$); at $N = 24$ virtually every decision is late. The cascade supports $N = 24$ stations (p95 155 ms, 0.2 % misses, $\rho = 0.87$) and degrades gracefully: at $N = 32$ only 13.6 % of decisions miss the deadline, and even in overload at most the offloaded half is late, because clear cases are still decided locally. With an edge twice as slow, the cascade supports 8 stations and edge-only 4. The cascade thus raises the number of stations per edge server by 1.5–2× while delivering 99 % of the edge-only F1, and reduces the mean uplink per station from 821 to 504 kbit/s (39 %). The load model (5) with $\mu(8) = 96$ frames/s predicts saturation at $N_{\max} = \mu/(fo) \approx 19$ stations for edge-only and ≈ 37 for the cascade ($o = 0.52$), consistent with the simulated knees between 16–24 and 24–32 stations; at light load it underestimates utilisation because batches are small ($\mu(1) = 64$). It is a useful first-order sizing rule: one CPU edge server per $\approx \mu/(fo)$ stations.

Constrained uplinks. At 1 Mbit/s a single 14–26 kB offload needs 110–210 ms of transmission, which alone exhausts the 200-ms budget. Edge-only decisions are therefore always late (100 % misses for every N), the cascade misses essentially its offloaded fraction (51–53 % for $N \geq 8$), and only station-only inspection meets the deadline. On such links the station model must take all time-critical decisions; offloaded verdicts remain

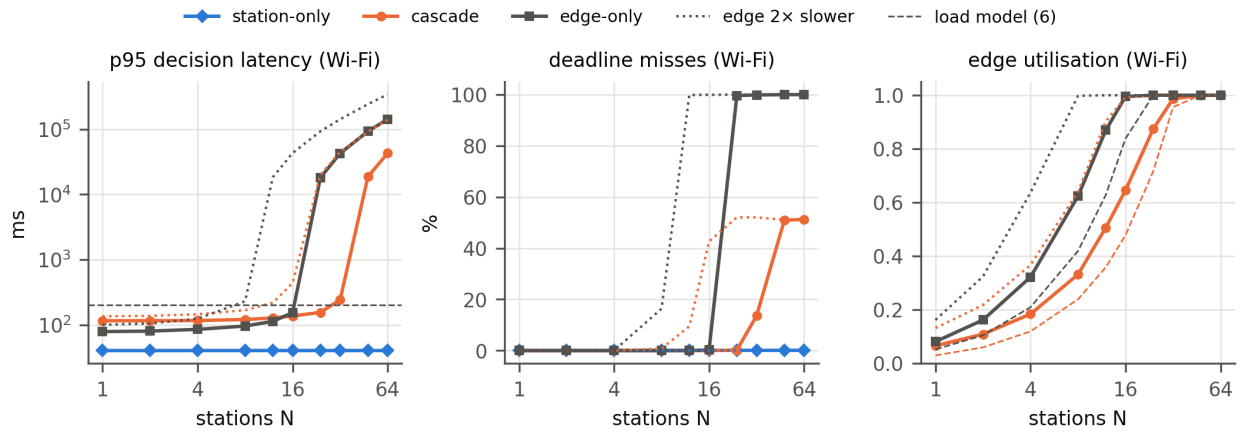


Fig. 4. Validated trace-driven simulation on the Wi-Fi link (10 seeds × 60 s per point): p95 decision latency (dashed line: 200-ms frame period), deadline misses, and edge utilisation versus the number of stations. Dotted: edge server twice as slow; thin dashed: load model (5) with $\mu(8)$.

useful as delayed quality records, or the payload must shrink (smaller crop, stronger compression).

VII. Discussion and Limitations

Data. The evaluation uses MVTec AD textures as a proxy for fabrics because openly downloadable, pixel-annotated textile data sets are scarce. Carpet and grid are the closest analogues of woven fabric and are also the hardest categories for the station model; leather, tile, and wood are easier. Validation on images captured at the seaming machines themselves—which the system logs automatically, including every flagged frame—is the most important next step.

Thresholds. The LOO rule is label-free and works well for the edge model on four of five textures, but it fails on grid, where the normal-score distribution of ten shots does not cover the variability of the test images. More shots help (Section VI-A); an operator-in-the-loop update that adds confirmed false alarms to the memory bank is a simple remedy that the logging already supports.

Testbed and simulation. The live testbed runs on one laptop, whose power management throttled the processor by 4–8× under sustained load; we therefore validated a trace-driven simulator on short live runs and used it for scaling. The simulator models service times, link serialisation, delays, queueing and batching, but not radio effects (contention, retransmissions, jitter bursts), broker load at very large scale, or operating-system jitter on real station hardware. The station CPU is one core of a laptop processor rather than an embedded board; to ease replication, the repository contains a standalone station benchmark that needs only NumPy, SciPy, and OpenCV. Energy consumption was not measured.

Scope. The system detects anomalies; it does not name the defect type. Classifying flagged frames, for example with a small supervised model trained incrementally on the logged snapshots, would turn the inspection log into a labelled data set over time. Security of the MQTT deployment (TLS, authentication, access control lists) is supported by standard brokers but was disabled in the experiments.

VIII. Conclusion

We presented FabricInspect-IoT, a few-shot, label-free fabric inspection service for networked seaming stations, and characterised it as an IIoT system. A fabric is learned from ten defect-free frames in about a second; a 6 kB station model and a ResNet-18 nearest-neighbour edge model are fitted together, and both obtain their decision thresholds by leave-one-out scoring of the shots, without any defect example. With these thresholds, the edge model reaches F1 0.935 at 4.3 % false alarms on four of five MVTec AD textures, while the fifth (grid) exposes the limits of ten-shot calibration. Confidence-gated offloading retains 99 % of the edge-only F1 while sending only half of the frames to the edge and lowers the station-only false-alarm rate from 28 % to 8 %; the gate adapts its offload rate to the fabric (23–81 %) using only the label-free thresholds. A trace-driven simulator, validated against a live testbed with a production MQTT broker, shows that on Wi-Fi a single CPU edge server meets a 200-ms decision deadline for 24 stations with the gate versus 16 without it (8 vs. 4 for a server twice as slow), while uplink traffic per station drops by 39 %; on a 1-Mbit/s uplink only locally decided frames meet the deadline, which makes the station model indispensable there. Retained MQTT messages distribute a new fabric model to any number of stations within 15 ms on Wi-Fi.

Future work will validate the system on images logged at the seaming machines, port the station model to the machines' Android controllers, add operator feedback to refine the LOO thresholds, and extend the gate to adapt α to the instantaneous edge load and uplink quality.

Author Contributions

N. Palvannazirova: conceptualization, methodology, software, investigation, data curation, writing—original draft preparation. A. Waseem: software, formal analysis, validation, writing—review and editing. N. Lobasevich: formal analysis, visualization, writing—review and editing. P. Butovskiy: supervision, methodology, validation, writing—review and editing. M. Allamov: investigation, resources, validation, writing—review and editing.

ACKNOWLEDGMENT

The authors used Claude (Anthropic), an AI assistant, to help implement the software, script and run the experiments, analyse the results, and draft the text of this manuscript. All experiments were executed on the authors' hardware and all reported numbers are produced by the released code. The authors reviewed and edited all content and take full responsibility for it.

REFERENCES

[1] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, 2008.

[2] R. Carrilho, E. Yaghoubi, J. Lindo, K. Hambarde, and H. Proença, "Toward automated fabric defect detection: A survey of recent computer vision approaches," *Electronics*, vol. 13, no. 18, p. 3728, 2024.

[3] Y. Wu, X. Zhang, and F. Fang, "Automatic fabric defect detection using cascaded mixed feature pyramid with guided localization," *Sensors*, vol. 20, no. 3, p. 871, 2020.

[4] T. Zhou, J. Zhang, H. Su, W. Zou, and B. Zhang, "EDDs: A series of efficient defect detectors for fabric quality inspection," *Measurement*, vol. 172, p. 108885, 2021.

[5] J. Zhang, J. Jing, P. Lu, and S. Song, "Improved MobileNetV2-SSDLite for automatic fabric defect detection system based on cloud-edge computing," *Measurement*, vol. 201, p. 111665, 2022.

[6] R. Machado, L. A. M. Barros, V. Vieira, F. D. d. Silva, H. Costa, and V. Carvalho, "Textile defect detection using artificial intelligence and computer vision—a preliminary deep learning approach," *Electronics*, vol. 14, no. 18, p. 3692, 2025.

[7] M. Nasim, R. Mumtaz, M. Ahmad, and A. Ali, "Fabric defect detection in real world manufacturing using deep learning," *Information*, vol. 15, no. 8, p. 476, 2024.

[8] H. Xu, C. Liu, S. Duan, L. Ren, G. Cheng, and B. Hao, "A fabric defect segmentation model based on improved Swin-Unet with Gabor filter," *Appl. Sci.*, vol. 13, no. 20, p. 11386, 2023.

[9] Q. Ye, J. Hou, Y. Yang, J. Xue, and Z. Liu, "OW-DLN: A novel open-world decoupled learning network for fabric defect detection," *Pattern Recognit.*, vol. 175, p. 113112, 2026.

[10] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, "Industrial internet of things: Challenges, opportunities, and directions," *IEEE Trans. Ind. Informat.*, vol. 14, no. 11, pp. 4724–4734, 2018.

[11] Z. Zhang, C. Qiu, X. Wang, F. Da, and S. Gai, "Dynamic multi-scale feature distillation with global–local decoupling for fabric anomaly detection," *Measurement*, vol. 273, p. 121132, 2026.

[12] H. Ma, Z. Li, H. Fan, X. Zheng, J. Yan, and R. Hu, "Phased noise enhanced multiple feature discrimination network for fabric defect detection," *Eng. Appl. Artif. Intell.*, vol. 149, p. 110480, 2025.

[13] F. An, S. Cao, S. Ma, D. Shu, B. Han, W. Li, and R. Liu, "Causality-inspired surface defect detection by transferring knowledge from natural images," *Eng. Appl. Artif. Intell.*, vol. 142, p. 109984, 2025.

[14] O. Rippel, P. Mertens, and D. Merhof, "Modeling the distribution of normal data in pre-trained deep features for anomaly detection," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, 2021, pp. 6726–6733.

[15] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR Int. Workshops Challenges*, ser. LNCS. Springer, 2021, pp. 475–489.

[16] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 14 298–14 308.

[17] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[18] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[19] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, 2020.

[20] L. Li, K. Ota, and M. Dong, "Deep learning for smart industry: Efficient manufacture inspection system with fog computing," *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4665–4673, 2018.

[21] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD—a comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9584–9592.

[22] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038–1059, 2021.

[23] R. A. Light, "Mosquitto: Server and client implementation of the MQTT protocol," *J. Open Source Softw.*, vol. 2, no. 13, p. 265, 2017.

[24] A. Kumar and G. K. H. Pang, "Defect detection in textured materials using Gabor filters," *IEEE Trans. Ind. Appl.*, vol. 38, no. 2, pp. 425–440, 2002.

[25] A. Bodnarova, M. Bennamoun, and S. Latham, "Optimal Gabor filters for textile flaw detection," *Pattern Recognit.*, vol. 35, no. 12, pp. 2973–2991, 2002.

[26] N. Alruwais, E. Alabdulkreem, K. Mahmood, R. Marzouk, M. Assiri, A. A. Abdelmageed, S. Abdelbagi, and S. Drar, "Hybrid mutation moth flame optimization with deep learning-based smart fabric defect detection," *Comput. Electr. Eng.*, vol. 108, p. 108706, 2023.

[27] K. Zhou, J. Jia, W. Wu, M. Qian, and Z. Xiang, "Space-depth mutual compensation for fine-grained fabric defect detection model," *Appl. Soft Comput.*, vol. 172, p. 112869, 2025.

[28] W. Jiao, X. Cai, Y. Tian, and M. Zuo, "Dual-image differential transformer for periodic lace surface defect detection," *Eng. Appl. Artif. Intell.*, vol. 148, p. 110386, 2025.

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[30] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.

[31] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proc. 22nd Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2017, pp. 615–629.

[32] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.

[33] OASIS, "MQTT version 3.1.1," OASIS Standard, 2014, [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html.

[34] K. Cura, N. Rintala, T. Kamppuri, E. Saarimäki, and P. Heikkilä, "Textile recognition and sorting for recycling at an automated line using near infrared spectroscopy," *Recycling*, vol. 6, no. 1, p. 11, 2021.

[35] J. Li, Q. Liu, H. Niu, and X. Wang, "Identification of textile fiber composition in waste textiles using improved CNN and near-infrared spectroscopy," *J. Ind. Textiles*, vol. 55, 2025.

[36] Ö. Öztürk, R. Brönnimann, and P. Modregger, "Defect detection in glass fabric reinforced thermoplastics by laboratory-based X-ray scattering," *Compos. Part B Eng.*, vol. 252, p. 110502, 2023.

[37] J. Silvestre-Blanes, T. Albero-Albero, I. Miralles, R. Pérez-Llorens, and J. Moreno, "A public fabric database for defect detection methods and results," *Autex Res. J.*, vol. 19, no. 4, pp. 363–374, 2019.